

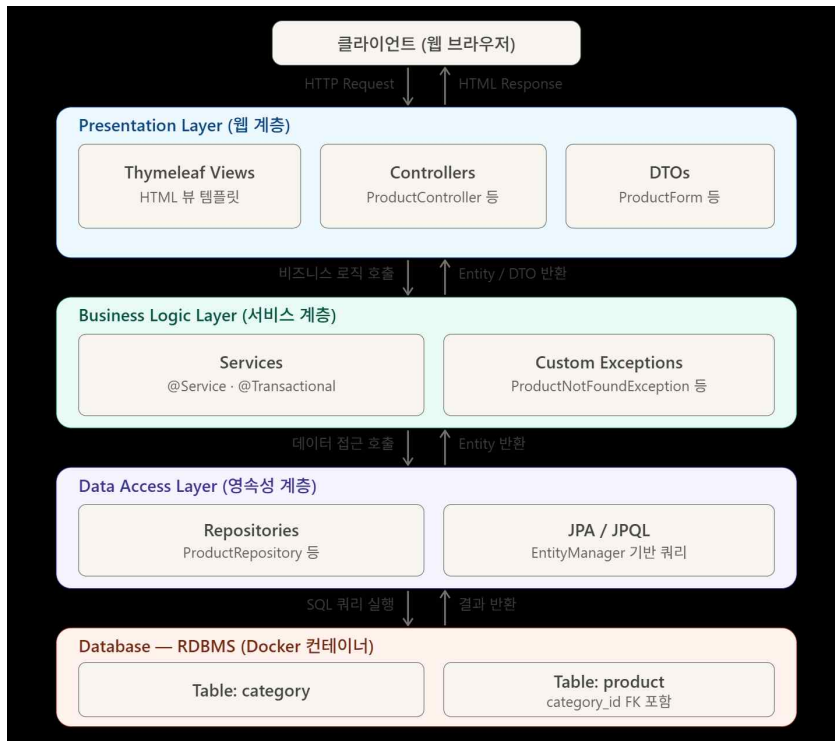
Spring MVC + JPA 상품 관리 시스템

학 번: 2371263

성 명: 원재연

GitHub: <https://github.com/inejy/helloSpringJpa>

1. 아키텍처 다이어그램



1. Client (클라이언트)

- 사용자가 브라우저를 통해 URL에 접속하거나 검색, 품 제출 등의 액션을 수행합니다.

2. Presentation Layer (프레젠테이션 계층 - Controller & View)

- Controller: 클라이언트의 HTTP 요청을 매핑(@GetMapping, @PostMapping)받아 적절한 로직으로 분기합니다. 품 입력값의 유효성을 검사(@Valid, BindingResult)하고, 예외 발생 시 에러 메시지를 뷰로 전달합니다.
- View (Thymeleaf): Controller가 넘겨준 Model 데이터를 기반으로 동적인 HTML 페이지를 렌더링하여 클라이언트에게 응답합니다.

3. Business Logic Layer (비즈니스 계층 - Service)

- 애플리케이션의 핵심 비즈니스 규칙(예: 카테고리 중복 검사, 삭제 제한 조건 검사 등)을 처리합니다.
- @Transactional을 통해 데이터베이스 작업의 트랜잭션 경계를 설정하고 데이터를 안전하게 관리합니다.

4. Data Access Layer (데이터 접근 계층 - Repository)

- 순수 데이터베이스 접근 로직을 담당합니다. JPA의 EntityManager를 사용하여 데이터베이스에 엔티티(Entity)를 저장, 수정, 삭제, 조회(JPQL)합니다.

5. Database (데이터베이스)

- Docker Compose로 실행된 RDBMS에 상품(Product)과 카테고리(Category) 테이블이 구축되어 실제 데이터를 영구적으로 저장합니다.

2. 카테고리 기능 코드 분석

2.1 Repository 계층: 연관 데이터 집계 최적화

```
java
// CategoryRepository.java
public long countProductsByCategoryId(Long categoryId) {
    return em.createQuery(
        "SELECT COUNT(p) FROM Product p WHERE p.category.id = :id",
        Long.class)
        .setParameter("id", categoryId).getSingleResult();
}
```

- 스칼라 타입 반환 쿼리: 카테고리 삭제 가능 여부를 판단하기 위해 `Product` 엔티티 전체를 `getResultList()`로 메모리에 로드하는 대신, JPQL의 집계 함수 `COUNT(p)`를 사용하여 단일 `Long` 타입 스칼라 값만 반환받도록 구현했습니다. 이를 통해 DB I/O와 애플리케이션 메모리 사용량을 최소화했습니다.

2.2 Service 계층: 트랜잭션 경계 설정 및 무결성 검증

```
java
// CategoryService.java
@Transactional
public void deleteCategory(Long id) {
    long count = categoryRepository.countProductsByCategoryId(id);
    if (count > 0) {
        throw new IllegalStateException("상품 " + count + "개가 연결되어 있어 삭제할 수 없습니다.");
    }
    categoryRepository.delete(id);
}
```

- 트랜잭션 롤백과 참조 무결성 방어: `@Transactional` 블록 내에서 `delete`를 수행하기 전, `count` 값을 검증합니다. DB 레벨의 외래 키(FK) 제약 조건에 의해 `DataIntegrityViolationException`이 발생하기 전에, 애플리케이션 레벨에서 선제적으로 `IllegalStateException`이라는 Unchecked Exception을 던집니다. 이로 인해 트랜잭션은 안전하게 롤백되며 시스템의 참조 무결성이 유지됩니다.

2.3 Controller 계층: 데이터 바인딩 및 예외 매핑

```
java
// CategoryController.java
@PostMapping("/create")
```

```

public String createCategory(@Valid @ModelAttribute CategoryForm
categoryForm, BindingResult bindingResult) {
    if (bindingResult.hasErrors()) return "categoryForm";

    try {
        categoryService.createCategory(categoryForm.getName());
    } catch (DuplicateCategoryException e) {
        bindingResult.rejectValue("name", "duplicate", e.getMessage());
        return "categoryForm";
    }
    return "redirect:/categories";
}

```

- 검증 흐름 제어 및 필드 에러 주입: Spring MVC의 `DataBinder`가 폼 파라미터를 `CategoryForm` 객체에 바인딩하고 `@Valid`를 통해 JSR-380 표준 검증을 수행합니다. 비즈니스 로직(Service)에서 발생한 `DuplicateCategoryException`을 Catch한 경우, `bindingResult.rejectValue()`를 호출하여 특정 필드(`name`)에 커스텀 에러 코드를 명시적으로 주입합니다. 이를 통해 Thymeleaf 뷰 레이어(`th:errors`)가 해당 예외를 폼 검증 에러와 동일한 생명주기로 처리할 수 있게 합니다.

3. 상품 검색 및 필터 기능 코드 분석

3.1 ProductRepository: 동적 JPQL 및 지연 로딩 초기화

```

java
// ProductRepository.java
public List<Product> search(String keyword, Long categoryId) {
    String jpql = "SELECT p FROM Product p LEFT JOIN FETCH p.category
WHERE 1=1";

    if (keyword != null && !keyword.trim().isEmpty()) {
        jpql += " AND p.name LIKE :keyword";
    }
    if (categoryId != null) {
        jpql += " AND p.category.id = :categoryId";
    }

    TypedQuery<Product> query = entityManager.createQuery(jpql, Product.class);
    if (keyword != null && !keyword.trim().isEmpty())
        query.setParameter("keyword", "%" + keyword + "%");
    if (categoryId != null) query.setParameter("categoryId", categoryId);
}

```

```

return query.getResultList();
}

```

- **동적 쿼리 조립 (Dynamic Query)**: 파라미터의 Null 여부에 따라 조건절이 변경되어야 하므로 `WHERE 1=1`을 기준으로 삼아 `AND` 절을 동적으로 추가하는 방식을 채택했습니다. `setParameter()`를 사용함으로써 조립된 JPQL은 Hibernate 내부에서 파라미터화된 쿼리(`PreparedStatement`)로 변환되어 SQL Injection 공격을 방어합니다.

- **N+1 문제 원천 차단 (FETCH JOIN)**: `Product` 엔티티 조회 시 연관된 `Category` 필드에 대해 `LEFT JOIN FETCH`를 선언했습니다. 이는 엔티티 매핑이 `FetchType.LAZY`로 되어 있더라도, 단일 쿼리 실행 시점에 카테고리 데이터까지 강제로 영속화(Initialization)하여 영속성 컨텍스트에 올립니다. 결과적으로 뷰(View) 렌더링 단계에서 프록시 객체 초기화로 인한 추가 쿼리(N+1)나 `LazyInitializationException`이 발생하지 않습니다.

3.2 ProductController: 쿼리 스트링 매핑 및 UI 상태 동기화

```

java
// ProductController.java
@GetMapping
public String listProducts(@RequestParam(value = "search", required = false)
String search,
                           @RequestParam(value = "categoryId", required = false)
Long categoryId, Model model) {

    List<Product> products = productService.searchProducts(search, categoryId);

    model.addAttribute("products", products);
    model.addAttribute("categories", categoryService.getAllCategories());
    model.addAttribute("searchKeyword", search);
    model.addAttribute("selectedCatId", categoryId);
    return "productList";
}

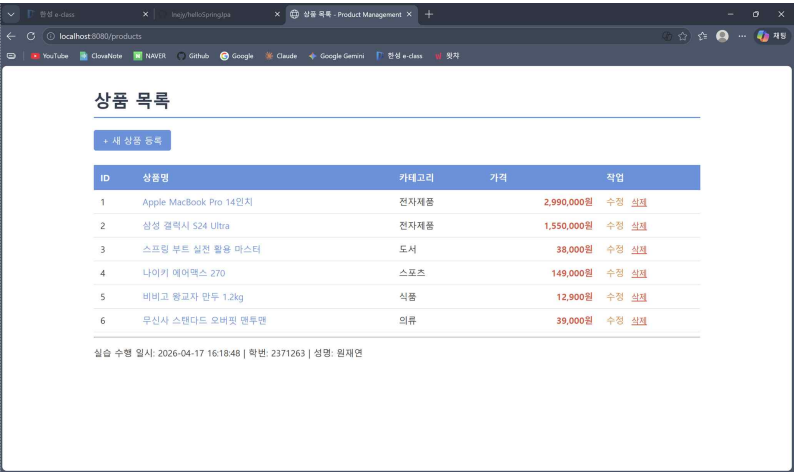
```

- **선택적 파라미터 바인딩**: HTTP GET 요청의 쿼리 스트링을 `@RequestParam(required = false)`로 매핑하여, 클라이언트가 일부 조건(예: 검색어만 입력 또는 카테고리만 선택)을 누락하더라도 `MissingServletRequestParameterException` 없이 `null`로 안전하게 바인딩되도록 컨트롤러 계층을 설계했습니다.

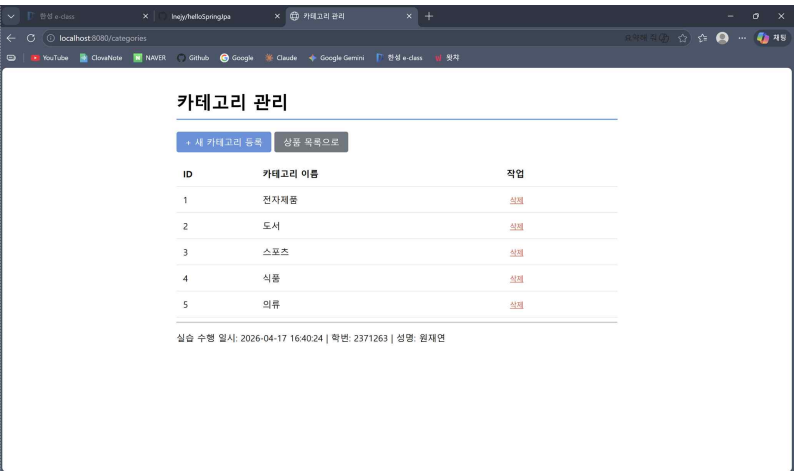
- **무상태(Stateless) 프로토콜의 상태 보존**: HTTP의 무상태 특성으로 인해 렌더링이 끝나면 검색 조건이 소멸됩니다. 이를 해결하기 위해 요청으로 들어온 `search`와 `categoryId` 변수를 다시 `Model`에 적재합니다. Thymeleaf 템플릿은 이 모델 데이터를 읽어 `<input th:value>`와 `<option th:selected>`에 주입함으로써, 사용자가 검색 후에도 이전의 입력 상태를 유지하는 동기화 메커니즘을 구현했습니다.

4. 실행 화면 스크린샷

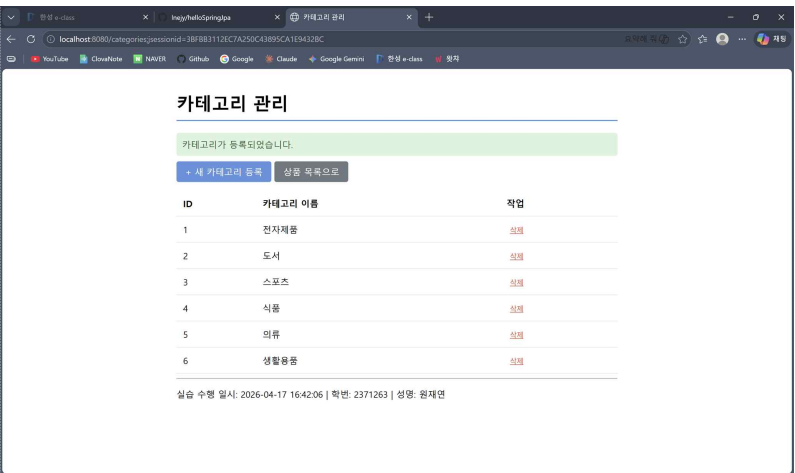
4.1 기반 코드 실행 화면(productList)



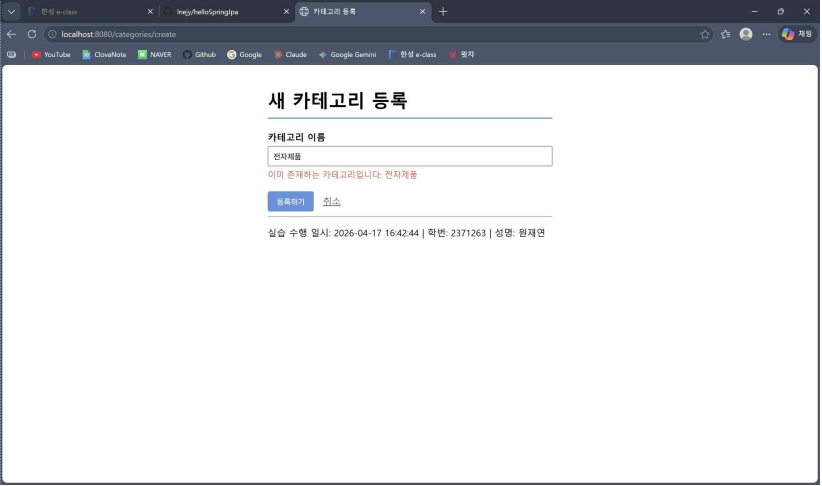
4.2 카테고리 목록 화면



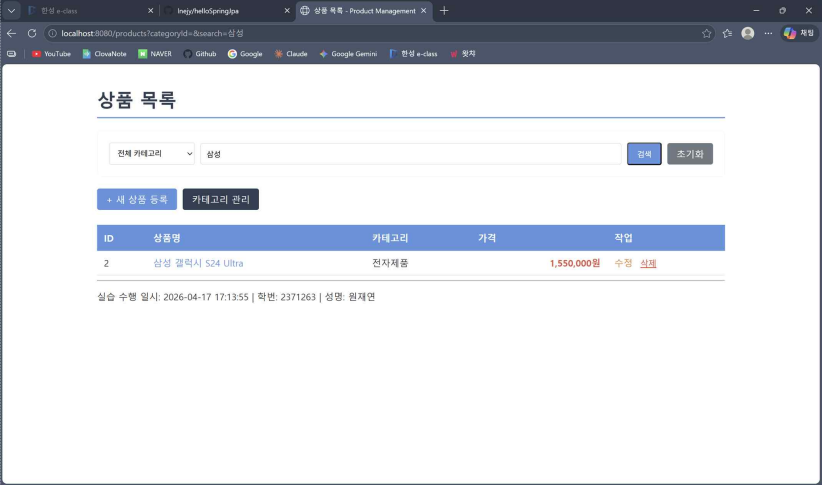
4.3 카테고리 등록 완료 화면



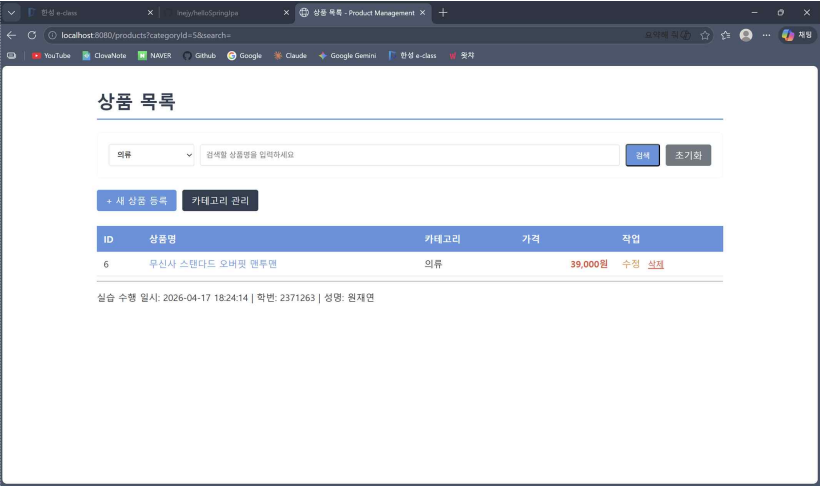
4.4 카테고리 중복 오류 메시지 화면



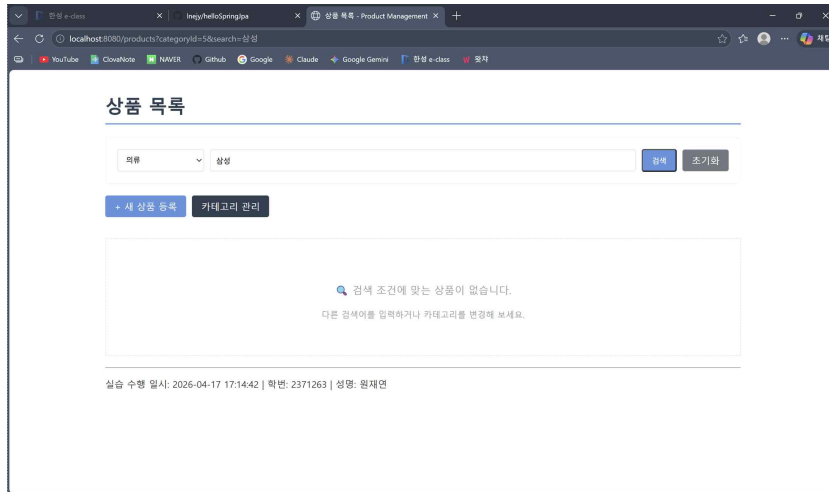
4.5 상품명 키워드 검색 결과 화면



4.6 카테고리 필터 적용 화면



4.7 검색 결과 없음 화면



5. 학습 소감 및 어려웠던 점

5.1 학습 소감

스프링 MVC 아키텍처의 실질적 이해: 이전까지는 이론으로만 접했던 Controller, Service, Repository 계층의 역할 분리를 직접 코드로 구현하며 그 필요성을 체감할 수 있었습니다. 특히, 프레젠테이션 계층(Controller)은 HTTP 요청과 응답 처리에만 집중하고, 비즈니스 규칙(Service)과 데이터 접근(Repository)을 분리함으로써 코드의 가독성과 유지보수성이 크게 향상된다는 점을 깊이 이해하게 되었습니다.

객체 지향적인 데이터베이스 관리 (JPA): 과거 SQL 쿼리를 직접 작성하던 방식과 달리, JPA와 영속성 컨텍스트를 활용하여 Java 객체(Entity) 상태를 변경하는 것만으로 데이터베이스가 동기화되는 과정이 매우 인상적이었습니다. 특히 DTO(Data Transfer Object)와 Entity를 엄격히 분리하여, 화면에서 요구하는 데이터 스펙의 변경이 데이터베이스 설계에 영향을 주지 않도록 시스템을 유연하게 구축하는 방법을 배웠습니다.

방어적 프로그래밍의 중요성 인식: 카테고리 중복 생성 방지나 삭제 시 참조 무결성 검증 로직을 구현하면서, "사용자는 항상 예측 불가능한 행동을 할 수 있다"는 전제하에 애플리케이션 단에서 꼼꼼하게 예외를 차단하는 방어적 프로그래밍의 중요성을 깨달았습니다.

5.2 어려웠던 점 및 극복 과정

지연 로딩(Lazy Loading)과 N+1 문제의 딜레마: 초기 구현 시, 상품 목록을 렌더링하는 과정에서 카테고리 이름을 출력할 때마다 추가적인 SELECT 쿼리가 발생하는 'N+1 문제'를 마주했습니다. 성능 저하의 원인이 프록시 객체 초기화에 있음을 학습한 후, JPQL의 LEFT JOIN FETCH를 도입하여 단일 쿼리로 연관된 데이터를 한 번에 메모리에 적재하도록 쿼리를 튜닝하여 문제를 해결했습니다. 이를 통해 ORM 사용 시 발생할 수 있는 성능 이슈와 최적화 기법을 직접 경험할 수 있었습니다.

동적 검색 쿼리 작성 및 파라미터 처리: 사용자가 '검색어만 입력', '카테고리만 선택', '둘 다 입력'하는 등 다양한 시나리오에 대응하는 쿼리를 작성하는 것이 까다로웠습니다. 초기에는 조건문이 복잡하게 얹혔으나, WHERE 1=1 구문을 활용하여 쿼리를 동적으로 안전하게 조립

하고, setParameter()로 바인딩하여 SQL Injection 위험까지 방어하는 구조로 로직을 개선했습니다.

UI 상태 유지와 폼 에러 핸들링 (State Retention): HTTP의 무상태성(Stateless) 때문에 폼 검증 실패 시나 검색 직후에 사용자가 입력했던 값이 날아가는 현상을 겪었습니다. 이를 해결하기 위해 BindingResult를 활용하여 검증 에러를 잡아내고, 요청받은 파라미터를 다시 Model에 담아 Thymeleaf 뷰로 전달하여 <input>과 <select> 태그의 상태를 복원하는 메커니즘을 구현하며 웹 요청의 전체적인 흐름과 생명주기를 명확히 이해하게 되었습니다.

6. 채점 (자기 평가)

배점	채점 항목	확인 방법 (채점자)	자기 평가 ☞□
【GitHub · 기반 코드 실행】			
2점	GitHub URL + commit 확인	URL 클릭 접속, commit 5회 이상 + 메시지 규칙 GitHub에서 직접 확인	2점 / 메시지 규칙 준수, commit 횟수 달성
2점	날짜/시간/학번 표시	모든 스크린샷에 날짜·시간·학번·성명 포함 여부	2점 / 모든 스크린샷에 출력 성공
【기능 구현 – Category 관리】			
3점	Category 목록 + 등록폼 동작	GET /categories, GET /categories/create 화면 정상 렌더링	3점 / 카테고리 목록 화면과 새 카테고리 등록 정상 렌더링
3점	Category 등록 처리 + 검증	@Valid 오류 메시지, 정상 등록 후 목록 리다이렉트	3점 / 등록 완료 메시지와 카테고리가 목록에 정상 추가 후 리다이렉트
2점	Category 중복 방지	동일 이름 재등록 시 폼에 오류 메시지 표시	2점 / 동일 이름 등록 시 오류 메시지 표시
3점	Category 삭제 + 연결 상품 예외	정상 삭제 동작, 연결 상품 있을 때 오류 Flash 메시지	3점 / 자식 데이터가 있을 때 예외 처리
【기능 구현 – 상품 검색】			
3점	상품명 키워드 검색	JPQL LIKE 검색 동작, 결과 없음 메시지 표시 확인	3점 / 삼성 키워드 검색 시 존재, LG 검색 시 존재 X
2점	카테고리 드롭다운 필터	카테고리 선택 시 해당 상품만 표시, 선택 상태 유지	2점 / 의류 카테고리 선택 시 알맞은 상품만 표시
【코드 분석 보고서】			
5점	GitHub URL + 아키텍처 다이어그램	URL 보고서 표지에 기재, 다이어그램 직접 작성 여부	5점 / 1페이지에 URL 및 다이어그램 존재
5점	기능 코드 분석 서술	추가 구현 코드의 동작 원리를 본인 이해 기반으로 서술 (복불 여부 판단)	4점 / 이유와 설명을 서술함 다만 완벽한 정답은 아닐 수 있음
30점	합 계	29점	